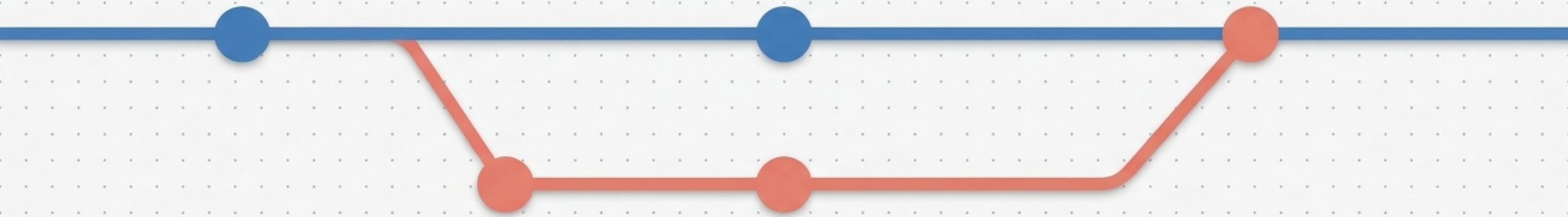
Controle de Versão: Do Zero ao GitOps

O manual definitivo de Git, GitHub e infraestrutura declarativa para engenharia moderna.



Gemini Notebook

O Motor Local versus A Nuvem Colaborativa

	Git 	GitHub 
O que é?	Programa de linha de comando (criado por Linus Torvalds).	Serviço na nuvem que hospeda repositórios.
Onde roda?	Na sua máquina, offline.	Em servidores remotos na internet.
Papel Principal	Gravar a história dos arquivos em um banco de dados local.	Viabilizar revisão em equipe (Pull Requests, Issues, Actions).
Alternativas	**Subversion** (SVN), **Mercurial** (raros hoje).	**GitLab**, **Bitbucket**, **Gitea**.



Em **Kubernetes**, o repositório **Git** é a "fonte da verdade" onde os manifestos moram, não importando se está no **GitHub** ou **GitLab**.

Gemini Notebook

O Modelo Mental Fundamental: As Três Áreas

Diretório de Trabalho



Seus arquivos atuais. Onde você edita o código.

`git add arquivo`

Área de Preparação

A "caixa" onde você coloca o que vai entrar no próximo próximo registro.

`git commit -m 'Mensagem'`

Repositório Local

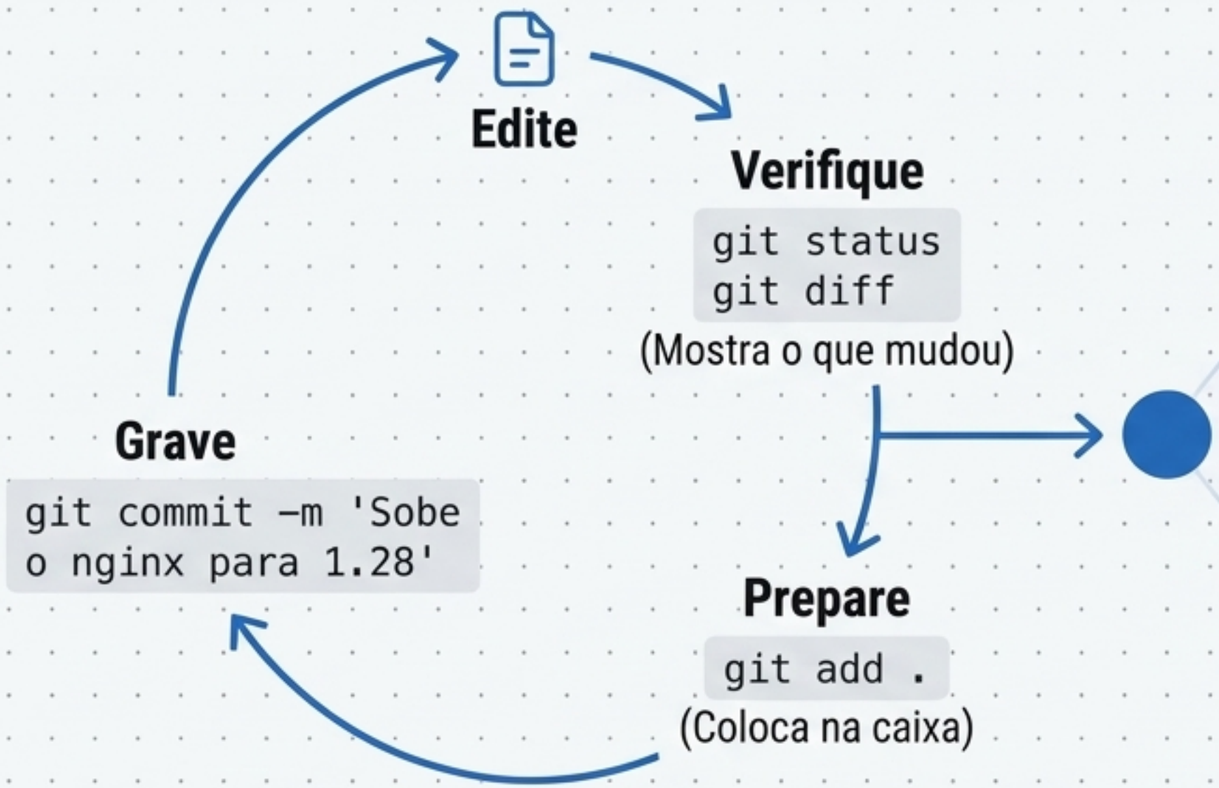


A história permanente. Onde os commits vivem.

Por que três áreas? Para agrupar mudanças lógicas. Você pode editar 5 arquivos, mas usar o `**add**` em apenas 2 para fazer um commit focado.

Gemini Notebook

O Ciclo de Trabalho e a Anatomia de um Commit



Anatomia do Commit

Hash: `0d28f4b`
(ID único gerado pelo conteúdo)

Pai (Parent):
(Aponta para o commit anterior)

Autor/Data:
(Quem e quando)

Mensagem:
(O "porquê". Mensagens devem ser curtas e no imperativo.)

⚠ **O Guarda-Costas `.gitignore`**
Lista padrões de arquivos para o Git ignorar (logs, senhas).
Precisa existir antes do arquivo ser commitado!

Gemini Notebook

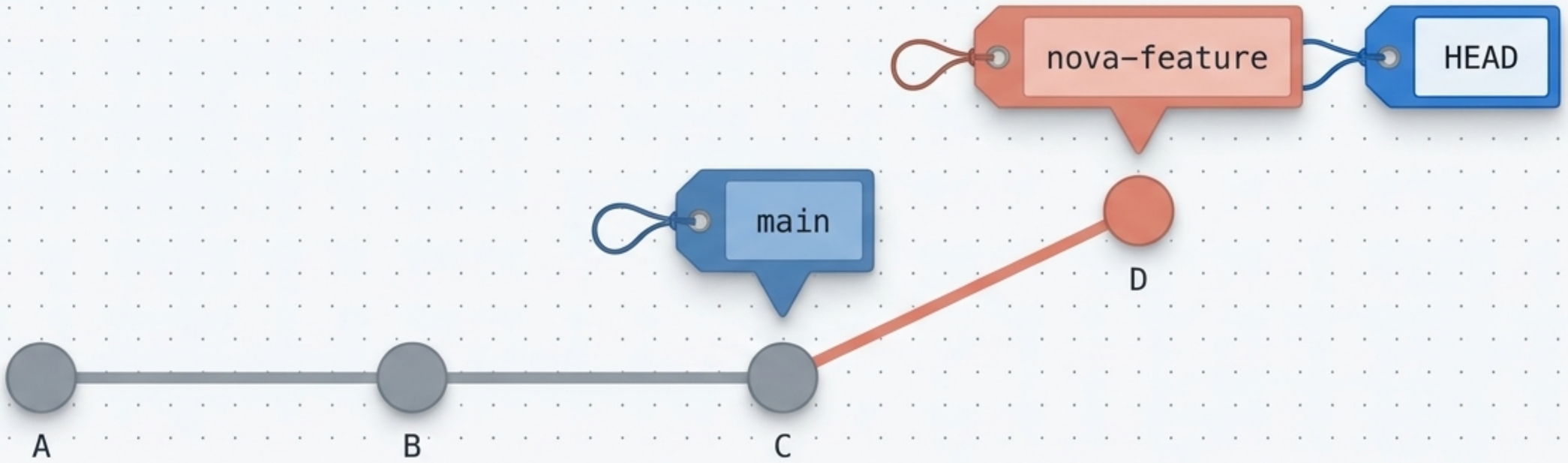
Kit de Primeiros Socorros: Desfazendo Erros sem Pânico

O que aconteceu?	O que você quer?	Comando de Salvação
Editei um arquivo, mas me arrependi.	Descartar a edição.	`git restore arquivo`
Dei `git add` no arquivo errado.	Tirar da 'caixa' (Staging), mantendo a edição.	`git restore --staged arquivo`
Esqueci algo no último commit ou errei a mensagem.	Corrigir o commit (se não foi publicado).	`git commit --amend`
Quero voltar no tempo e apagar os últimos commits.	Desfazer tudo (Cuidado!).	`git reset --hard HEAD~1`
Preciso desfazer um erro que já enviei para a equipe.	Criar um commit novo que inverte o erro.	`git revert <hash>`

A Rede de Segurança: O Reflog. Se você fez um `reset --hard` por acidente, o Git guarda o histórico do seu ponteiro. Use `git reflog` para achar o hash antigo e voltar à vida!

Gemini Notebook

Realidades Paralelas: Trabalhando com Branches



O que é?

Uma branch não copia arquivos; é apenas um ponteiro móvel para um commit. Por isso é instantânea e barata.

Para que serve?

Isolar ideias. Teste uma mudança em uma branch sem quebrar a `main`. Se falhar, apague a branch e nada aconteceu.

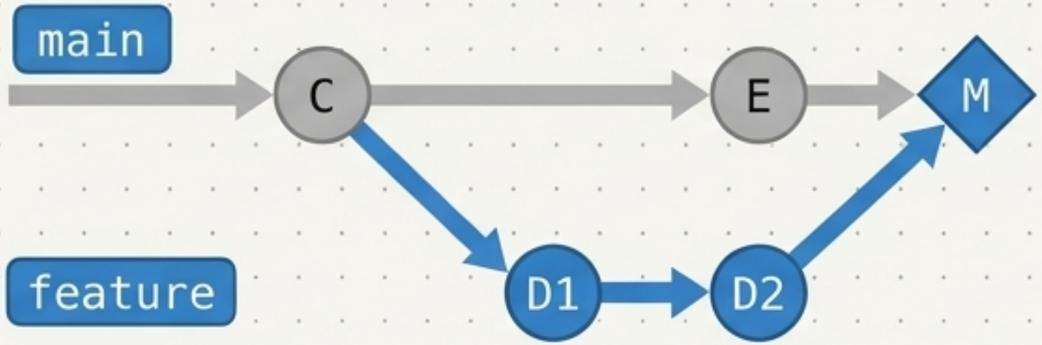
```
git switch -c nova-feature  (Cria e entra na branch)

git switch main             (Volta para a main)
```

Gemini Notebook

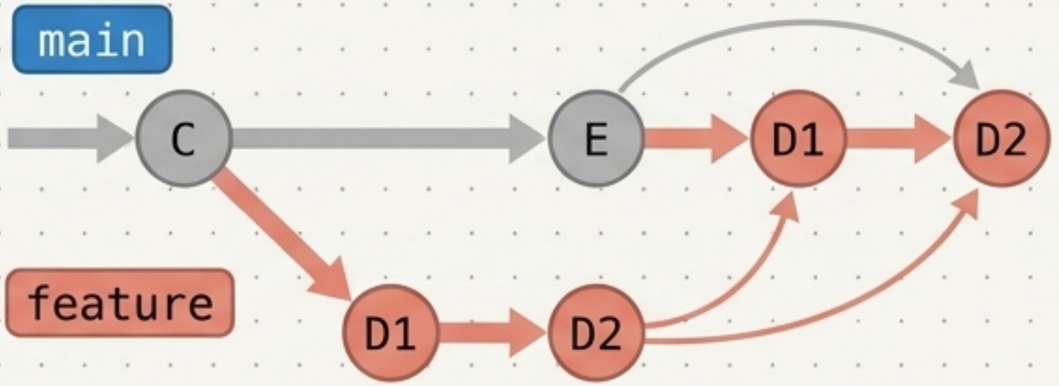
Unindo Histórias: O Confronto entre *Merge* e *Rebase*

Git Merge



- Preserva a história exatamente como aconteceu.
- Cria um commit de união com dois pais.
- **Veredito**: Seguro para qualquer situação, essencial para integrar código ao tronco principal.

Git Rebase

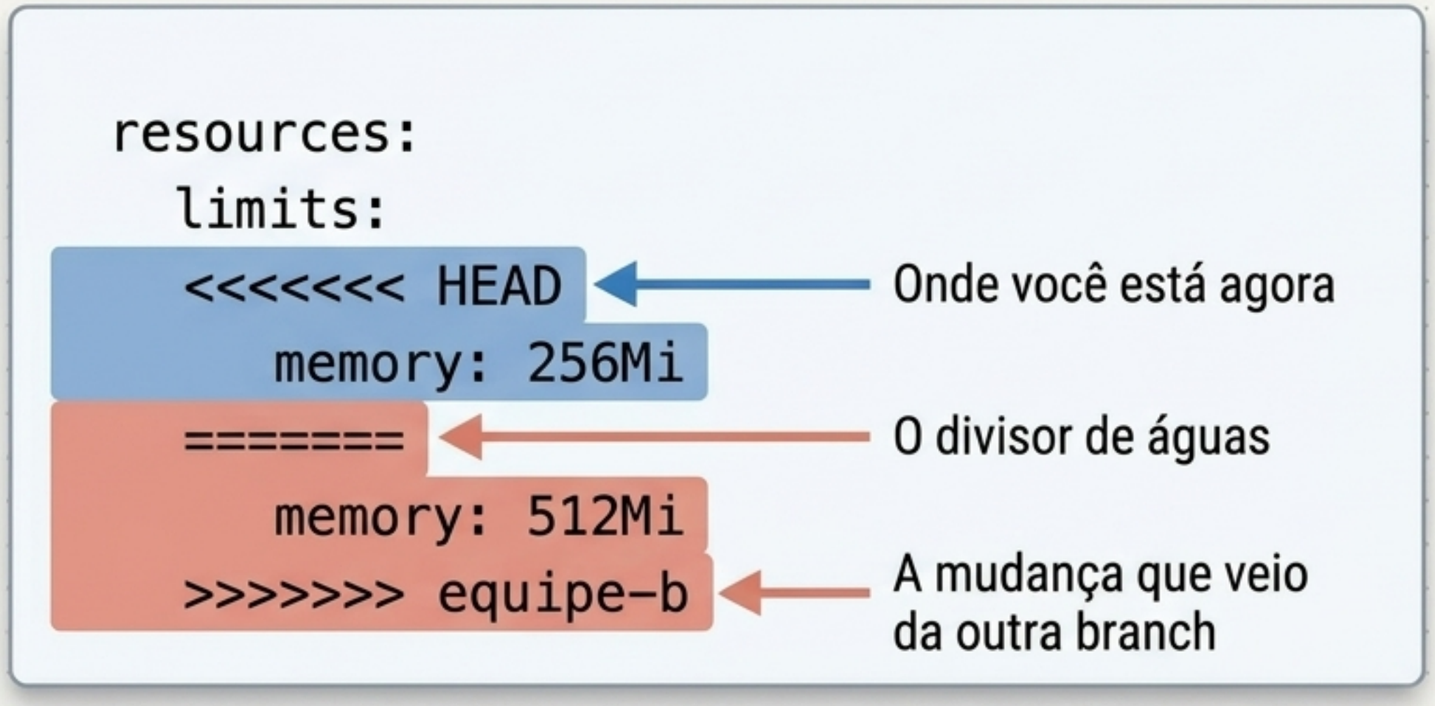


- Reaplica seus commits sobre a ponta da `main`.
- A história fica em linha reta, mas reescreve os hashes originais.

A Regra de Ouro: NUNCA faça rebase de commits que outras pessoas já baixaram. Use apenas na sua branch local para atualizar com a `main` antes de um Pull Request.

Gemini Notebook

Anatomia de um Conflito: Não entre em Pânico

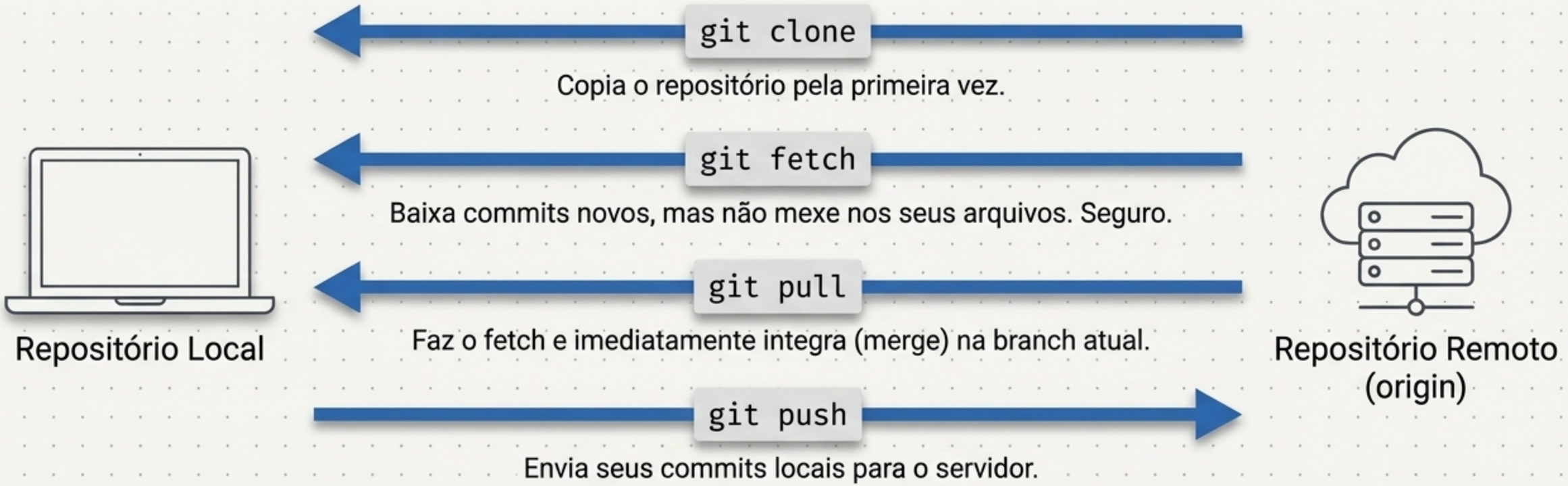


1. **Entenda:** Conflito não é erro. O Git só não quer adivinhar qual versão manter.
2. **Edite**: Apague as marcações (`<<<<<<<` , `=======` , `>>>>>>>`) e deixe apenas o código final desejado.
3. **Grave**: Rode `git add` no arquivo e ` `git commit` para selar a paz.

Pânico total? `git merge --abort` cancela tudo e devolve ao estado anterior.

Gemini Notebook

O Ecossistema Remoto: Conectando Local e Origin



O Famoso Push Rejeitado

Erro: `fetch first` (O remoto tem commits que você não tem localmente).
Solução: Faça um `git pull`, resolva eventuais conflitos, e tente o `push` novamente.
Nunca use o parâmetro `--force` **sobrepondo o trabalho alheio!**

Gemini Notebook

A Regra de Ouro de Segurança no Git



NUNCA coloque senhas, tokens, chaves privadas ou arquivos kubeconfig no repositório.

Why? (Por quê?)

Bots são rápidos	Base64 não é criptografia	Apagar não resolve
Bots varrem repositórios públicos em minutos atrás de credenciais expostas.	Em Kubernetes, um Secret em YAML expõe o dado a quem puder ler o arquivo.	Fazer um novo commit apagando a senha é inútil; ela continuará acessível nos commits antigos.

The Fix

Se vazou, considere comprometido. Revogue a credencial no provedor imediatamente.
No dia a dia, use arquivos `.env` listados no `.gitignore` ou ferramentas como SOPS.

Autenticação

Para a sua máquina falar com o GitHub, use chaves SSH (`ssh-keygen`) ou tokens pessoais.
O GitHub não aceita mais senhas de conta.

Gemini Notebook

A Engrenagem da Equipe: Pull Requests e Automação

Branch Local

Você publica sua branch de funcionalidade.

```
git push
```

Pull Request (PR)



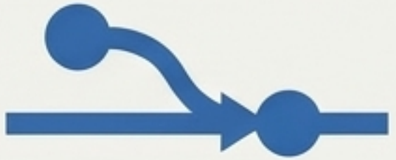
Pedido formal para integrar seu código. É onde ocorrem discussões de linha a linha e revisões.

GitHub Actions (CI)



Robôs entram em ação. Pipelines rodam testes (ex: `yamllint`) para bloquear código quebrado.

Merge Protegido



Após aprovação humana e testes verdes, o código entra na `main` de forma segura.

Conceito-Chave

O Pull Request não é um comando do Git, é uma ferramenta do provedor (GitHub, GitLab) para proteger a branch principal e fomentar a colaboração assíncrona.

Gemini Notebook

GitOps: O Repositório como a Única Fonte da Verdade

Repositório Git



(O estado desejado)



Agente GitOps
(Argo CD / Flux)

Ninguém roda `kubectl apply` manualmente na produção.

Cluster Kubernetes



(O estado real)

Auditoria Automática	Rollback à prova de falhas	Autocorreção
		
Todo `pull request` serve como `log` **documentado** e **imutável** de quem alterou a infraestrutura e por quê.	Quebrou a produção? Basta um `git revert` do último commit. O agente sincroniza a reversão em segundos.	Se alguém alterar o cluster `na mão via terminal`, o agente detecta a divergência e desfaz, forçando a obediência ao Git.

Gemini Notebook

O Mapa de Bolso: Comandos Essenciais do Git

O Básico Local

`git init` / `git clone` : Cria ou copia repositório

`git status -sb` / `git diff` : Onde estou e o que mudou

`git add .` & `git commit -m` : Prepara e grava

`git log --oneline` : Vê a história

Branches e Integração

`git switch -c nova` : Cria e entra na branch

`git switch main` : Muda de branch

`git merge nova` : Junta histórias

Desfazendo (Salva-Vidas)

`git restore arquivo` : Descarta edição

`git reset --soft HEAD~1` : Desfaz commit, mantém arquivo na caixa

`git revert HEAD` : Desfaz criando commit novo (seguro para nuvem)

Remoto e Colaboração

`git fetch` & `git pull` : Baixa / Baixa e integra

`git push -u origin main` : Envia commits

`git stash` : Guarda trabalho pela metade temporariamente

O segredo não é decorar todos os comandos, é entender em qual 'área' você está e para onde quer mover seus arquivos.

Gemini Notebook